

## TP 2 — GitOps avec ArgoCD : DevHub Campus

### Conteneurisation, Helm et livraison continue tirée par Git — vue du développeur

Cours d'architecture logicielle & déploiement — M2 Ingénierie Web

2025–2026

#### Table des matières

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Avant-propos .....                                                                    | 2  |
| Contexte métier .....                                                                 | 2  |
| Suite logique du TP 1 .....                                                           | 2  |
| Compétences visées .....                                                              | 2  |
| Modalités .....                                                                       | 3  |
| Plateformes supportées .....                                                          | 3  |
| Conventions du document .....                                                         | 3  |
| Comment lire ce poly .....                                                            | 4  |
| Pourquoi GitOps, et pourquoi ArgoCD côté développeur .....                            | 4  |
| Le modèle <i>push</i> et ses limites .....                                            | 4  |
| Le modèle <i>pull</i> (GitOps) .....                                                  | 4  |
| Le piège à éviter .....                                                               | 5  |
| Le même geste, deux paradigmes .....                                                  | 5  |
| Architecture cible .....                                                              | 6  |
| PARTIE I — Fondations .....                                                           | 8  |
| Étape 0 — Outillage .....                                                             | 8  |
| Étape 1 — Comprendre GitOps avant d'écrire la moindre ligne .....                     | 10 |
| Étape 2 — Le vocabulaire d'ArgoCD .....                                               | 11 |
| Étape 3 — Containeriser un service .....                                              | 11 |
| Étape 4 — Écrire le chart Helm du service .....                                       | 12 |
| PARTIE II — GitOps en action .....                                                    | 14 |
| Étape 5 — Installer ArgoCD et déclarer la première Application .....                  | 14 |
| Étape 6 — Le pattern <i>App of Apps</i> .....                                         | 15 |
| Étape 7 — ApplicationSet : des environnements de preview par branche .....            | 17 |
| PARTIE III — Maîtrise et synthèse .....                                               | 18 |
| Étape 8 — Drift, rollback, hooks, sync waves .....                                    | 18 |
| Étape 9 — Sécuriser et observer ArgoCD lui-même .....                                 | 20 |
| Étape 10 — Comparer les outils GitOps ( <i>bonus, au-delà des 7h</i> ) .....          | 21 |
| Étape 11 — Synthèse obligatoire : « <i>ArgoCD, et la prod alors ?</i> » .....         | 22 |
| Boss final ( <i>bonus, pour les binômes qui terminent largement en avance</i> ) ..... | 23 |
| Annexe A — Pour aller plus loin .....                                                 | 24 |
| Annexe B — Pièges fréquents .....                                                     | 24 |
| Annexe C — Critères d'évaluation .....                                                | 25 |

# Avant-propos

## Contexte métier

DevHub Campus est une plateforme interne fictive d'une école d'ingénieurs. Elle regroupe trois services métier que des équipes différentes développent en parallèle :

- annuaire-service — répertoire des étudiants, enseignants et intervenants ;
- planning-service — emplois du temps, salles, créneaux ;
- notif-service — émission d'événements vers Slack / email / push (mocké en TP).

Chaque service vit dans son propre dépôt Git, possède son chart Helm, et est livré par un développeur — vous — sur un cluster Kubernetes mutualisé. La consigne de la direction technique est claire : « *on ne veut plus voir un `kubectl apply` en production. Tout passe par Git.* »

Vous êtes un développeur de la plateforme. Votre mission : prendre en main une démarche GitOps avec ArgoCD, mettre vos services sous pilotage automatique, et obtenir des environnements de preview éphémères à chaque branche.

## Suite logique du TP 1

Le TP SalleEnFrance vous a mis en posture d'équipe DevOps : provisionner, sécuriser, monter une pipeline CI/CD qui `kubectl apply`. Le TP 2 vous met côté développeur : vous ne *poussez* plus le cluster, vous décrivez l'état souhaité dans Git et le cluster *tire* ce qu'il faut.

**L'enjeu pédagogique central de ce TP est là.** Tout au long du document, des encarts *Avant (TP 1) / Maintenant (TP 2)* matérialisent le contraste sur le **même geste** opérationnel. À la fin, vous devez être capable d'argumenter, pour chaque opération de la vie d'un cluster, « *pourquoi je le ferais autrement qu'au TP 1 — et pourquoi c'est mieux (ou pire).* »

| TP 1 — SalleEnFrance                           | TP 2 — DevHub Campus                                    |
|------------------------------------------------|---------------------------------------------------------|
| Posture ops / SRE                              | Posture développeur                                     |
| <code>kubectl apply</code> en CI               | ArgoCD lit Git en continu                               |
| Manifests YAML écrits à la main                | Chart Helm, <code>values.yaml</code> par environnement  |
| Un environnement (dev)                         | Multi-environnements + previews par branche             |
| Outils : Freelens, Terraform, cert-manager     | Outils : ArgoCD UI, Helm, ApplicationSet                |
| Concept central : <i>Pod, Service, Ingress</i> | Concept central : <i>Application, App of Apps, Sync</i> |

## Compétences visées

À la fin du TP, vous saurez :

1. Expliquer la différence entre une livraison *push* (CI qui `apply`) et une livraison *pull* (GitOps).
2. Écrire un chart Helm minimal pour un microservice (Chart.yaml, values, templates paramétrés).
3. Installer ArgoCD sur un cluster et y déclarer une Application qui suit un dépôt Git.
4. Mettre en place le pattern App of Apps pour orchestrer plusieurs services depuis un dépôt unique.
5. Créer des environnements de preview éphémères grâce à un ApplicationSet adossé aux branches d'un dépôt.

6. Diagnostiquer un drift (état réel  $\neq$  état désiré), comprendre la différence entre auto-sync, self-heal et prune.
7. Sécuriser ArgoCD côté plateforme : AppProject, RBAC, notifications, observabilité.
8. Caractériser ce qu'ArgoCD **ne fait pas** et nommer les briques complémentaires nécessaires pour une vraie chaîne de prod.

## Modalités

- Rendu en binôme, sur un dépôt Git par binôme, avec un rapport RAPPORT.md.
- Durée cible : 1 journée (7 heures) sur le tronc commun (étapes 0 à 9 et 11). Les étapes 10 et le *Boss final* sont des bonus pour les binômes qui avancent vite.
- L'accompagnement formateur est oral et à la demande. Posez les questions à voix haute, montrez l'écran.
- Le squelette de départ vous est fourni dans le dossier argocd/devhub-campus/ du dépôt de cours : services pré-codés, charts en squelette, manifestes ArgoCD avec marqueurs # TODO. **Forkez-le, ne le clonez pas.**

## Plateformes supportées

Ce TP fonctionne identiquement sur macOS, Windows et Linux. Le cluster Kubernetes utilisé en TP est démarré par un seul `make cluster-up` : un conteneur sur votre machine, aucune VM, aucun hyperviseur, aucun KVM. La distribution Kubernetes sous-jacente est un détail d'implémentation du Makefile — l'enjeu du TP est ArgoCD, pas le cluster.

| Plateforme    | Setup requis                                                                                                                                        |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| macOS         | Docker Desktop <i>ou</i> OrbStack. Les commandes du Makefile s'exécutent dans le terminal.                                                          |
| Windows 10/11 | Docker Desktop avec <b>backend WSL2 obligatoire</b> + une distro Ubuntu WSL2. Les commandes du Makefile s'exécutent dans WSL2, pas dans PowerShell. |
| Linux         | Docker Engine.                                                                                                                                      |

Sur Windows, l'édition du fichier `hosts` se fait dans `C:\Windows\System32\drivers\etc\hosts` (depuis un PowerShell *administrateur*), pas dans le `/etc/hosts` de WSL2 — sinon les hosts ne sont pas résolus par votre navigateur Windows.

## Conventions du document

**Objectif.** ce qui doit être atteint à la fin de l'étape.

**Contraintes.** règles non-négociables (sécurité, naming, ressources).

**Décomposition suggérée.** un découpage en sous-tâches avec, pour chacune, les concepts à consulter dans la documentation officielle. Vous n'êtes pas obligé de la suivre — c'est une béquille pour ne pas rester bloqué.

**Livrable.** le fichier ou la sortie attendue dans votre dépôt.

**Validation.** ce que vous devez voir / vérifier dans l'UI ArgoCD pour considérer l'étape réussie.

**Pièges.** erreurs classiques qui vous feront perdre du temps.

**Défi bonus.** extension non-évaluée qui pousse l'étape plus loin. Pour les binômes qui finissent vite ou veulent se challenger.

## Comment lire ce poly

Vous êtes en M2 — vous lisez ce document comme vous voulez, en choisissant votre mode :

| Mode                    | Pour qui                                 | Ce que vous lisez                                                                       |
|-------------------------|------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Brief de mission</b> | Vous voulez vous débrouiller seul.       | Objectif, Contraintes, Livrable, Validation. Le reste : à éviter, ça gâche le plaisir.  |
| <b>Pas à pas</b>        | Vous voulez avancer sans rester coincés. | Tout, y compris la <i>Décomposition suggérée</i> . Lisez aussi la doc officielle citée. |
| <b>Défi</b>             | Vous avancez vite et voulez plus.        | Idem brief + tous les <i>Défis bonus</i> , et le <i>Boss final</i> en clôture.          |

Quel que soit le mode, le formateur évalue **la même grille** (annexe C). Les défis bonus ne donnent pas de points en plus mais sont retenus pour la mention.

## Pourquoi GitOps, et pourquoi ArgoCD côté développeur

### Le modèle *push* et ses limites

Dans le TP 1, votre pipeline GitHub Actions exécutait `kubectl apply` en fin de course. Ce modèle s'appelle *push* : la CI a les droits sur le cluster, elle pousse les changements. Il marche très bien tant que :

- une seule pipeline a les droits ;
- les développeurs n'ont pas besoin de visibilité sur l'état du cluster ;
- on ne déploie qu'un seul environnement à la fois.

Dès que vous voulez :

- offrir aux devs **leur** environnement de preview sans leur donner les droits cluster ;
- voir d'un coup d'œil *qu'est-ce qui tourne en ce moment, version par version* ;
- annuler une mise en prod en quelques secondes sans rejouer la CI ;
- enregistrer dans l'historique Git **chaque** changement qui touche au cluster ;

...le modèle *push* devient inconfortable. C'est le terrain de jeu de GitOps.

### Le modèle *pull* (GitOps)

Le principe tient en une phrase : **un agent dans le cluster lit en continu un dépôt Git et fait converger l'état du cluster vers ce qui y est décrit.**

- Le dépôt Git devient la *source de vérité*. Si ce n'est pas dans Git, ce n'est pas dans le cluster.
- La CI ne touche plus au cluster. Elle se contente de produire l'image OCI et d'écrire un commit dans le dépôt de configuration.
- Un changement = un commit. Un retour arrière = un revert.
- L'agent (`argocd-application-controller`) détecte les écarts (*drift*) et peut les corriger automatiquement.

ArgoCD est l'un des deux agents les plus utilisés sur le marché (l'autre est Flux). Il a une UI graphique riche, ce qui en fait l'outil GitOps préféré des équipes produit / dev.

### Le piège à éviter

ArgoCD est souvent présenté comme *l'outil de prod GitOps*. C'est partiellement vrai mais trompeur pour un développeur.

En l'état, ArgoCD :

- ne fait **pas** de déploiement progressif (canary, blue/green) — il faut Argo Rollouts ;
- ne fait **pas** de validation de politiques (interdire un `:latest`, exiger un `securityContext`) — il faut Kyverno ou OPA Gatekeeper ;
- ne gère **pas** les secrets — il faut Sealed Secrets, External Secrets, ou Vault ;
- ne signe **pas** les images, ne vérifie **pas** les signatures — il faut cosign + une admission policy ;
- ne fait **pas** de disaster recovery applicatif (snapshots PVC, dump SGBD).

ArgoCD est un excellent **distributeur** d'état désiré. Ce n'est pas une chaîne de release complète. Ce TP vous fait sentir où s'arrête l'outil, et ce que cela implique pour la prod.

### Le même geste, deux paradigmes

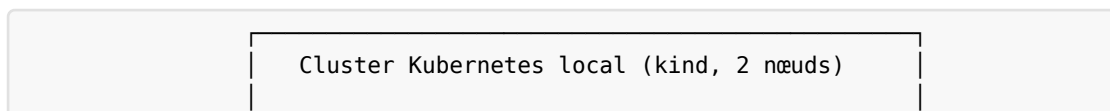
Avant d'écrire la moindre ligne, prenez le temps de lire ce tableau. C'est la colonne vertébrale du TP. Vous le retrouverez en filigrane dans chaque étape, et vous devrez en restituer une version commentée dans votre rapport final (étape 11).

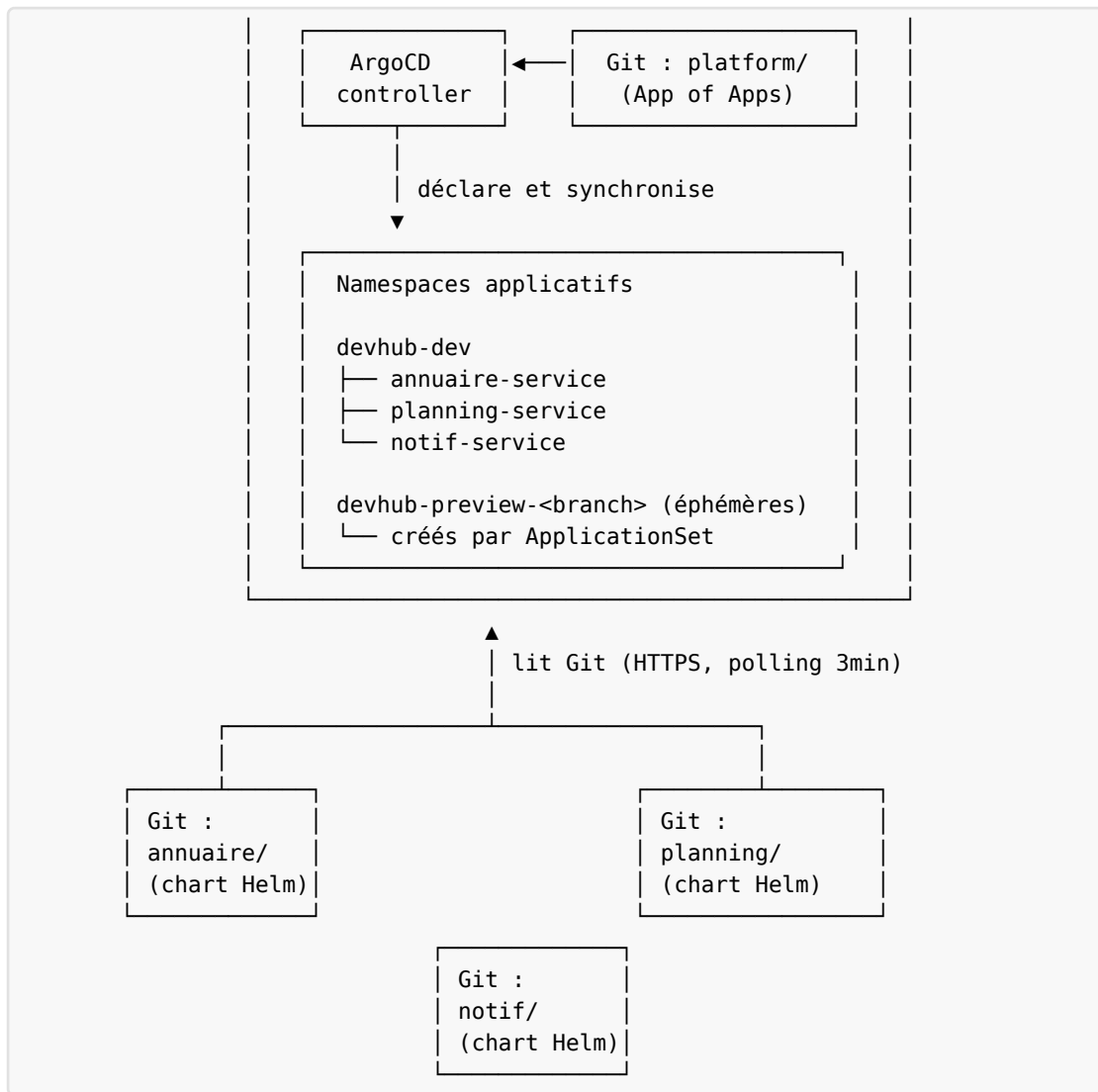
| Opération du quotidien                                          | Au TP 1 (Kubernetes « à la main »)                                                                               | Au TP 2 (Kubernetes piloté par ArgoCD)                                           |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Déployer un service pour la première fois                       | Écrire 5 YAML, <code>kubectl apply -f</code> , vérifier dans Freelens                                            | Commit du chart Helm dans Git, ArgoCD détecte, sync auto                         |
| Déployer une nouvelle version                                   | Modifier le tag dans le YAML, re- <code>kubectl apply</code> (ou via la CI)                                      | Commit qui change <code>image.tag</code> , c'est tout                            |
| Faire un rollback                                               | <code>kubectl rollout undo</code> ou relancer la pipeline avec l'ancien commit                                   | <code>git revert</code> du commit fautif, ArgoCD re-converge                     |
| Ouvrir un environnement de plus                                 | Copier le dossier <code>overlays/dev</code> en <code>overlays/staging</code> , créer un namespace, refaire le CI | Ajouter une Application dans le repo <code>platform/</code> . Fin.               |
| Donner un env perso à chaque dev                                | Quasiment impossible sans donner les droits cluster                                                              | ApplicationSet + branche Git = preview automatique                               |
| Voir ce qui tourne <i>en ce moment</i>                          | Freelens / <code>kubectl get all -A</code> / mémoire collective                                                  | UI ArgoCD : un coup d'œil, on voit l'état et la version                          |
| Détecter qu'un dev a fait un <code>kubectl edit</code> en douce | Personne ne s'en aperçoit. Drift silencieux.                                                                     | ArgoCD passe en <code>OutOfSync</code> immédiatement.                            |
| Auto-réparer un drift                                           | Personne ne le fait. Ou un cron <code>kubectl apply</code> .                                                     | <code>selfHeal: true</code>                                                      |
| Donner les droits à un nouveau dev                              | Distribuer un kubeconfig, espérer qu'il ne fasse pas de bêtise                                                   | Compte ArgoCD avec rôle <code>developer</code> sur son AppProject                |
| Hotfix en urgence à 3h du matin                                 | SSH + <code>kubectl edit</code> (et on documentera demain...)                                                    | PR sur le repo, ArgoCD applique. Tracé.                                          |
| Auditer ce qui a changé sur le cluster sur 6 mois               | <code>kubectl get events</code> (rétention 1h) + Git log de la CI                                                | <code>git log</code> sur les repos <code>platform/</code> et services            |
| Re-déployer le cluster from scratch                             | Re-lancer Terraform + la CI + prier                                                                              | Re-lancer Terraform + 1 seul <code>kubectl apply</code> pour la root Application |
| Désinstaller un service                                         | <code>kubectl delete -f</code> (et espérer ne rien oublier)                                                      | Supprimer le fichier dans Git → prune propre                                     |
| Tester un changement risqué                                     | Sur le dev partagé. Croiser les doigts.                                                                          | Sur sa preview à soi, isolée par branche                                         |

Ce qu'il faut entendre derrière ce tableau : **ArgoCD n'apporte pas de nouvelles capacités à Kubernetes — il change qui agit, quand, et comment c'est tracé.** C'est un changement de *workflow*, pas un changement de plateforme.

**Question à garder en tête tout le TP.** Pour chaque opération du tableau ci-dessus, est-ce que la version « ArgoCD » est toujours meilleure ? Listez au moins deux opérations où elle est plus contraignante, et pourquoi.

## Architecture cible





| Composant                     | Rôle                                     | Outil                       | Vivant dans      |
|-------------------------------|------------------------------------------|-----------------------------|------------------|
| argocd-server                 | UI web et API                            | ArgoCD chart Helm           | namespace argocd |
| argocd-repo-server            | clone les dépôts Git, rend les templates | idem                        | idem             |
| argocd-application-controller | détecte le drift, réconcilie             | idem                        | idem             |
| platform repo                 | App of Apps + Application-Set            | Git seul                    | dépôt distant    |
| annuaire-service repo         | code + Dockerfile + chart Helm           | Git + GHCR                  | dépôt distant    |
| planning-service repo         | idem                                     | idem                        | idem             |
| notif-service repo            | idem                                     | idem                        | idem             |
| devhub-dev namespace          | environnement stable, branche main       | déclaré dans platform       | cluster          |
| devhub-preview-*              | environnements éphémères par branche     | générés par Application-Set | cluster          |

## PARTIE I — Fondations

### Étape 0 — Outillage

**Objectif.** disposer d'un poste capable d'opérer un cluster local et de dialoguer avec ArgoCD.

**Contraintes.**

- vous installez les outils vous-mêmes, sans ticket support ;
- vous documentez la version exacte de chaque outil dans `RAPPORT.md` ;
- sur Windows, **tous les outils ci-dessous s'installent dans WSL2**, pas dans Windows directement. Le docker que vous utilisez est celui exposé par Docker Desktop *via* WSL2.

Liste des outils requis :



| Outil                     | Pourquoi                                      | Source                    |
|---------------------------|-----------------------------------------------|---------------------------|
| Docker Desktop / OrbStack | runtime conteneur                             | docker.com / orbstack.dev |
| kubectl ≥ 1.30            | CLI K8s                                       | kubernetes.io             |
| kind ≥ 0.22               | cluster K8s local (vous l'aviez déjà au TP 1) | kind.sigs.k8s.io          |
| helm ≥ 3.14               | charts (ArgoCD, ingress-nginx)                | helm.sh                   |
| argocd CLI ≥ 2.11         | login, sync, debug                            | argo-cd.readthedocs.io    |
| git ≥ 2.40                | ...vraiment ?                                 | git-scm.com               |
| yq ≥ 4.40                 | manipuler des valeurs Helm en script          | mikefarah.gitbook.io/yq   |
| stern (optionnel)         | tail logs multi-pods                          | stern.dev                 |

### Décomposition suggérée.

1. Sur Windows : vérifiez que Docker Desktop tourne en backend WSL2 (*Settings* → *General* → *Use the WSL 2 based engine*) et que votre distro Ubuntu apparaît dans *Resources* → *WSL Integration*. **Sans ça, rien ne marchera.**
2. Forkez le repo devhub-campus/ fourni. Clonez votre fork dans votre WSL (jamais sur un montage Windows /mnt/c/... — c'est lent).
3. Lancez `make tools-check` : la cible doit afficher la version de chaque outil. Tout outil manquant doit être installé.
4. Provisionnez le cluster avec `make cluster-up`. La cible crée un cluster kind à deux nœuds, avec les ports 80/443 publiés sur l'hôte. **Vous ne reviendrez plus toucher au cluster directement après cette commande.**
5. Installez la CLI argocd si ce n'est pas déjà fait — *attention à l'homonymie avec d'autres CLI Argo (Workflows, Rollouts, Events)*. Vérifiez `argocd version --client`.
6. Lancez `make hosts-print` pour obtenir la liste des entrées à ajouter dans votre fichier `hosts` (chemin différent sur Windows, cf. *Plateformes supportées*).

**Livrable.** section *Outillage* du RAPPORT.md avec la sortie de `kubectl version --client`, `helm version`, `argocd version --client`.

### Pièges.

- la CLI `argocd` peut être confondue avec d'autres outils Argo (*Workflows, Rollouts, Events*). Vérifiez : `argocd version --client` doit afficher la version d'ArgoCD lui-même.
- sur Windows, oublier d'activer l'intégration WSL2 dans Docker Desktop : votre `docker version` répond, mais `docker run` reste bloqué indéfiniment. La résolution est dans *Resources* → *WSL Integration*.
- sur Windows, cloner sur un montage /mnt/c/... au lieu de ~/ dans WSL : `helm template` mettra 10 secondes au lieu de 100 millisecondes à cause du file-system bridge.

**Défi bonus.** scriptez un `make tools-check` plus exigeant : il doit non seulement vérifier la présence des outils, mais aussi leur version *minimale*. Sortez en erreur avec un message explicite si une version est insuffisante. Ajoutez-le à un *pre-commit hook* Git.

## Étape 1 — Comprendre GitOps avant d'écrire la moindre ligne

**Objectif.** être capable d'expliquer GitOps à un collègue qui n'en a jamais entendu parler, en restant techniquement précis.

### Décomposition suggérée.

1. Lisez les *Principles* de l'OpenGitOps working group ([opengitops.dev](https://opengitops.dev)) : *Declarative, Versioned and immutable, Pulled automatically, Continuously reconciled*. Mémorisez les quatre, vous les recroiserez en entretien.
2. Comparez en lisant la doc d'introduction d'ArgoCD et celle de Flux. Notez ce qui les distingue.
3. Faites votre propre schéma dans Excalidraw / draw.io. Pas de copie d'image.

**Livraison.** une section « *GitOps en 1 page* » dans `RAPPORT.md`, contenant :

1. un schéma personnel opposant le flux *push* (CI qui pousse) et le flux *pull* (agent qui tire) ;
2. le tableau ci-dessous à compléter ;
3. en deux phrases, votre prise de position : « *pour mes futurs projets perso, je commencerais par push ou par pull ? Pourquoi ?* ».

À compléter :

| Question                                                   | Push (kubectl apply en CI) | Pull (ArgoCD) |
|------------------------------------------------------------|----------------------------|---------------|
| Qui a les droits sur le cluster ?                          | ...                        | ...           |
| Où est l'historique des changements ?                      | ...                        | ...           |
| Que se passe-t-il si un dev modifie le cluster à la main ? | ...                        | ...           |
| Comment ajouter un environnement de plus ?                 | ...                        | ...           |
| Comment faire un rollback ?                                | ...                        | ...           |
| Combien de pipelines pour 30 services ?                    | ...                        | ...           |
| Qui voit <i>en direct</i> ce qui tourne ?                  | ...                        | ...           |

**Validation.** restituez votre position à voix haute au formateur. Si vous n'arrivez pas à la défendre, retournez lire avant de coder.

**Défi bonus.** ajoutez une troisième colonne *Flux* à votre tableau. Pour quels critères Flux gagne-t-il sur ArgoCD ? Lequel des deux retiendriez-vous pour une PME de 5 développeurs ? Pour un grand groupe avec 200 clusters ? Justifiez.

## Étape 2 — Le vocabulaire d'ArgoCD

**Objectif.** maîtriser le vocabulaire qui reviendra partout dans la suite. Sans ce socle, l'UI ArgoCD vous paraîtra confuse.

**Livable.** dans `RAPPORT.md`, une page de glossaire personnel où chacun des termes ci-dessous est défini *avec vos mots*, accompagné d'un « *exemple dans mon projet* » concret.

À définir :

| Terme                             | À ne pas confondre avec...                     |
|-----------------------------------|------------------------------------------------|
| Application (la ressource ArgoCD) | une application au sens K8s, ou au sens métier |
| AppProject                        | un projet Git, ou un namespace                 |
| Source                            | un repo Git, ou une URL de chart Helm          |
| Destination                       | un cluster + un namespace                      |
| Sync (manuel, auto, self-heal)    | un <code>kubectl apply</code>                  |
| Prune                             | un <code>kubectl delete</code>                 |
| App of Apps                       | un Helm chart de charts                        |
| ApplicationSet                    | un script qui boucle sur des Application       |
| Sync wave                         | une boucle de retry                            |
| Hook (PreSync, Sync, PostSync)    | un trigger Git                                 |

**Pièges.** beaucoup d'étudiants confondent *Application* (la ressource ArgoCD, vit dans le cluster) et le *contenu* applicatif qu'elle déploie. Soyez vigilants : tout au long du TP, on parle de *l'Application annuaire-dev*, pas de *l'application annuaire*.

**Défi bonus.** dessinez le diagramme d'état (state machine) d'une Application en répondant : quels événements la font passer de `OutOfSync` à `Synced` ? De `Healthy` à `Degraded` ? Quel champ du status indique chaque transition ?

## Étape 3 — Containeriser un service

**Objectif.** produire une image OCI propre pour **l'un des trois services**, à choisir par le binôme. Le squelette de code est fourni dans le dépôt initial du TP.

**Choix du service.**

- annuaire-service (Node.js, CRUD étudiants sur PostgreSQL — fourni avec sa migration) ;
- planning-service (Python FastAPI, calculs de créneaux) ;
- notif-service (Go, écoute un sujet et logge un événement).

**Contraintes** (héritées du TP 1, non négociables) :

1. multi-stage systématique ;
2. image finale  $\leq 200$  Mo ( $\leq 50$  Mo pour notif-service en Go) ;
3. utilisateur non-root ;
4. aucun secret en ENV ni en argument de build ;
5. tag d'image = SHA court du commit, jamais latest ;
6. exposition d'un endpoint /healthz joignable par les probes ;
7. LABEL org.opencontainers.image.source pointant vers le dépôt Git ;
8. l'image doit accepter une variable LOG\_LEVEL (debug, info, warn) et s'y conformer.

### Décomposition suggérée.

1. Lisez le README.md du service que vous avez choisi (dans le dépôt initial). Comprenez ce qu'il fait, et ce que ses dépendances (DB, etc.) attendent.
2. Faites tourner le service **en local hors Docker** avant de le containeriser. Si vous ne savez pas le démarrer à la main, votre image va échouer pour la même raison.
3. Écrivez le Dockerfile. Premier stage = compilation / install des dépendances. Second stage = image runtime, copie des artefacts, USER 1001.
4. Vérifiez la taille (docker images <image>), passez trivy dessus.
5. Publiez sur GHCR. Pour cela, créez un PAT GitHub avec scope write:packages et docker login ghcr.io.

**Livable.** Dockerfile à la racine du repo du service + une image publiée sur ghcr.io/<votre-handle>/<service>:<sha>.

**Validation.** un `docker run -p 8080:8080 -e LOG_LEVEL=debug <image>` répond 200 OK sur /healthz et logge un message *debug* au démarrage.

**Pièges.** ne pas confondre l'utilisateur du Dockerfile (USER 1001) avec celui qui apparaîtra dans le securityContext du Pod K8s — il faut que les deux s'accordent.

### Défi bonus.

- signez votre image avec `cosign sign --key cosign.key ghcr.io/<handle>/<service>:<sha>` et publiez la clé publique dans le repo. Documentez comment un consommateur peut vérifier la signature.
- ajoutez un HEALTHCHECK Docker dans votre Dockerfile. Pourquoi K8s l'ignore-t-il ? Que faut-il faire à la place ?

## Étape 4 — Écrire le chart Helm du service

**Objectif.** rendre votre service **packageable** au format Helm. Le chart sera ensuite consommé par ArgoCD.

### Contraintes.

- structure standard : Chart.yaml, values.yaml, templates/.

- paramètres exposés dans `values.yaml` au minimum : `image.repository`, `image.tag`, `replicaCount`, `service.port`, `env.LOG_LEVEL`, `ingress.enabled`, `ingress.host`.
- templates *a minima* : `deployment.yaml`, `service.yaml`, `ingress.yaml` (conditionnel), `_helpers.tpl` (préfixage de noms, labels communs).
- labels obligatoires sur chaque ressource : `app.kubernetes.io/name`, `app.kubernetes.io/instance`, `app.kubernetes.io/part-of: devhub-campus`, `app.kubernetes.io/managed-by: Helm`.
- probes : `readinessProbe` et `livenessProbe` HTTP sur `/healthz`.
- **trois fichiers de values dérivés** : `values-dev.yaml`, `values-staging.yaml`, `values-preview.yaml`. Ils ne contiennent que les **différences** par rapport à `values.yaml`.

### Décomposition suggérée.

1. `helm create chart` puis supprimez tout ce que vous ne comprenez pas. Mieux vaut un chart minimaliste qui marche qu'un chart par défaut bourré d'options que vous ne maîtrisez pas.
2. Écrivez `_helpers.tpl` en premier : le nom commun, le full name, les labels standards. Tout le reste s'appuie dessus.
3. Écrivez `deployment.yaml` ensuite. Faites un `helm template chart/` après chaque ajout pour valider que le rendu reste valide.
4. Ajoutez `service.yaml`, puis `ingress.yaml` (conditionnel via `{{- if .Values.ingress.enabled }}`).
5. Créez les trois fichiers `values-*.yaml`. Réfléchissez à ce qui change vraiment entre dev et preview : nombre de répliques, ressources, host de l'ingress...

**Livable.** dossier `chart/` à la racine du repo de votre service. La commande `helm template chart/ -f chart/values-dev.yaml` doit produire des manifests valides.

**Validation.** `helm lint chart/` retourne 0 `chart(s) failed. helm template ... | kubectl apply --dry-run=client -f -` passe.

### Pièges.

- oublier que `helm template` ne se connecte pas au cluster ; il rend localement les YAML. Si vous voyez des erreurs `Error: unable to recognize`, c'est un problème de manifests, pas d'ArgoCD.
- mettre toute la configuration dans `values.yaml` au lieu de la répartir par environnement — vous perdez tout l'intérêt du multi-env.

### Défi bonus.

- ajoutez un `values.schema.json` qui valide la structure des values (types, valeurs autorisées). `helm install` rejettera désormais un `replicaCount: -1` ou un `service.port: "huit-mille"`.
- écrivez un *test Helm* (`templates/tests/`) qui valide, après déploiement, que le service répond bien sur `/healthz`. Déclenchez-le avec `helm test`.

- faites en sorte que votre chart puisse aussi déployer un PodDisruptionBudget conditionnel (`pdb.enabled`).

## PARTIE II — GitOps en action

### Étape 5 — Installer ArgoCD et déclarer la première Application

**Avant (TP 1) / Maintenant (TP 2).** Au TP 1, déployer un service consistait à écrire les YAML, les commiter, et laisser la pipeline GitHub Actions exécuter `kubectl apply -k`. Ici, la pipeline ne touchera plus jamais au cluster : elle commit, et ArgoCD synchronise. Vous allez sentir la première bascule en faisant *un seul* `kubectl apply` dans cette étape (la *root* Application). Ce sera le dernier de tout le TP.

**Objectif.** installer ArgoCD dans le cluster du TP, accéder à son UI, et y déclarer une Application qui suit votre repo de service.

#### Contraintes.

- installation via le chart Helm officiel (`argo/argo-cd`), namespace dédié `argocd` ;
- exposition de l'UI via l'ingress du cluster, sur le host `argocd.devhub.local` ;
- HTTPS *insecure* toléré en TP (auto-signé) ;
- le mot de passe admin initial est récupéré depuis le secret `argocd-initial-admin-secret` puis **rotaté** (changement obligatoire, à documenter dans le rapport) ;
- la première Application :
  - se nomme `<service>-dev` (ex : `annuaire-dev`),
  - source = votre repo de service, chemin `chart/`, branche `main`, values `chart/values-dev.yaml`,
  - destination = cluster local, namespace `devhub-dev`,
  - sync policy = **manuel** d'abord (le bouton Sync est cliqué à la main),
  - puis basculée en auto-sync avec `selfHeal: true` et `prune: false` une fois validée.

#### Décomposition suggérée.

1. **Installez d'abord ingress-nginx** (prérequis pour exposer l'UI ArgoCD). Sur kind, deux subtilités : il faut un `nodeSelector: ingress-ready=true` et une `toleration` pour le taint du control-plane. Lisez la doc *kind ingress* avant de coder.
2. `helm repo add argo https://argoproj.github.io/argo-helm` puis `helm install argocd argo/argo-cd -n argocd --create-namespace`. Avant de lancer, regardez le `values.yaml` du chart en amont pour savoir ce que vous installez. **Un fichier `platform/argocd/values.yaml` est fourni dans le squelette** — relisez-le et complétez les TODO.
3. Activez l'ingress ArgoCD sur le host `argocd.devhub.local` (déjà en place dans le `values.yaml` fourni).
4. Récupérez le mot de passe initial (`kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d`).
5. Connectez-vous via la CLI (`argocd login argocd.devhub.local --insecure`) puis changez le mot de passe (`argocd account update-password`).

6. Complétez le manifeste Application dans `platform/apps/dev/<service>.yaml` (squelette fourni avec `# TODO`). Faites un `kubectl apply -f` à la main pour le créer. **C'est la dernière fois que vous tapez `kubectl apply` de tout ce TP.**

*Raccourci* : le Makefile du squelette propose `make argocd-install` qui exécute les étapes 1 et 2 en un appel. Vous pouvez l'utiliser, mais vous devez **savoir refaire à la main** — c'est l'objet de l'étape.

### Livrable.

- un dossier `platform/` dans un nouveau repo Git, contenant le manifeste de l'Application ;
- une capture d'écran de l'UI ArgoCD montrant l'Application *Healthy* et *Synced* ;
- dans `RAPPORT.md`, une comparaison entre `selfHeal: true` et `prune: true`, illustrée d'un exemple où *chacun* serait dangereux.

### Validation.

- dans l'UI ArgoCD, votre Application apparaît avec le statut *Synced + Healthy* ;
- en cliquant dessus, vous voyez l'arbre complet : `Deployment` → `ReplicaSet` → `Pods`, `Service`, `Ingress` ;
- les pods sont *Running* et *Ready* ;
- vous accédez à votre service via l'ingress (sur `<service>.devhub.local`).

### Pièges.

- confondre `Sync` (ArgoCD applique l'état du dépôt) et `Refresh` (ArgoCD relit le dépôt sans appliquer) ;
- laisser `prune: true` dès le départ : à la moindre erreur de chemin dans la source, ArgoCD supprime tout. Activez-le en connaissance de cause.

### Défi bonus.

- **Bootstrap récursif.** installez ArgoCD une seconde fois, mais cette fois *par ArgoCD lui-même* : créez une Application dont la source est le chart `argo/argo-cd`. ArgoCD se gère lui-même. Quel est l'avantage ? Quel est le piège (l'œuf et la poule) ?

## Étape 6 — Le pattern *App of Apps*

**Avant (TP 1) / Maintenant (TP 2).** Au TP 1, ajouter un service à la stack signifiait : créer ses YAML, l'ajouter dans le `kustomization.yaml`, relancer la pipeline. Cinq endroits à toucher pour un nouveau service. Ici, ajouter un service à la plateforme se résume à *un commit dans `platform/apps/dev/`* : un fichier Application de 15 lignes. La root le détecte, ArgoCD crée tout. Ce passage à l'échelle est le vrai gain quand on a 30 services et 6 équipes.

**Objectif.** ne plus déclarer les Application à la main. Un seul repo platform/ orchestre les trois services en environnement dev.

### Contraintes.

- structure du repo platform/ :
  - apps/dev/annuaire.yaml,
  - apps/dev/planning.yaml,
  - apps/dev/notif.yaml,
  - bootstrap/root-app.yaml (l'Application racine qui pointe vers apps/dev/).
- chaque Application enfant cible le chart Helm du repo de son service ;
- un AppProject devhub est créé avec :
  - liste blanche des dépôts Git autorisés (les trois repos de service),
  - liste blanche des destinations (cluster local + namespaces commençant par devhub-\*),
  - liste blanche des clusterResourceWhitelist (réduisez la surface).
- la création initiale se fait par `kubectl apply -f bootstrap/root-app.yaml` une fois — ensuite, tout passe par Git.

### Décomposition suggérée.

1. Écrivez d'abord l'AppProject devhub. Lisez la doc ArgoCD sur les AppProject : c'est votre brique de sécurité.
2. Migrez l'Application de l'étape 5 dans platform/apps/dev/annuaire.yaml (ou votre service). Ajoutez `project: devhub`.
3. Écrivez la *root* Application dans platform/bootstrap/root-app.yaml. Sa source = le repo platform/, son chemin = apps/dev/, son moteur de templating = *plain YAML* (pas Helm). La destination du root est le namespace argocd lui-même.
4. `kubectl apply -f bootstrap/root-app.yaml`. Vous devriez voir apparaître dans l'UI : la root, puis les enfants qu'elle a créés.
5. Demandez à un binôme voisin de pousser un commit sur le repo platform/ qui change un `replicaCount`. Observez la propagation sans toucher au cluster.

### Livrable.

- le repo platform/ complet, poussé sur GitHub ;
- une capture d'écran de l'UI ArgoCD montrant *quatre* Application : la racine et les trois enfants ;
- dans RAPPORT.md, une réponse argumentée à : « *Pourquoi le pattern App of Apps n'est-il pas équivalent à une simple `kubectl apply -f apps/dev/ ?` ».*

**Validation.** quand vous ajoutez un commit dans platform/apps/dev/ (par exemple : changer `replicaCount` dans le path d'une des sous-applications), ArgoCD détecte le changement, propose la sync, et applique sans que vous ayez touché au cluster.

### Pièges.



- oublier de mettre l'AppProject *avant* les Application qui y font référence — ArgoCD refusera leur création.
- mettre l'Application racine dans son propre dossier *cible* (récursion infinie). La racine pointe vers apps/dev/, pas vers bootstrap/.

**Défi bonus.** ajoutez à l'AppProject une roles et une sync window :

- un rôle developer qui n'autorise que les actions sync et get sur les Application du projet,
- une fenêtre de synchronisation qui **interdit** les syncs entre 18h et 8h du lundi au vendredi.

Démontrez que les deux fonctionnent (try argocd app sync à 22h).

## Étape 7 — ApplicationSet : des environnements de preview par branche

**Avant (TP 1) / Maintenant (TP 2).** Au TP 1, offrir un environnement de preview par branche aurait demandé : étendre les overlays Kustomize, scripter la création dynamique de namespaces dans la CI, gérer la suppression à la suppression de branche, et donner les droits cluster à la CI pour chaque preview. Plusieurs jours d'ingénierie pour un binôme. Ici, c'est l'étape la plus *visuellement marquante* du TP : un seul ApplicationSet, un *generator* Git, et toute la machinerie est en place. **C'est ici, et nulle part ailleurs, que vous mesurez vraiment ce qu'ArgoCD apporte côté dev.**

**Objectif.** chaque fois qu'un développeur pousse une branche feature/\* sur le repo de son service, un environnement de preview complet (namespace devhub-preview-<branche>, ingress sur <branche>.devhub.local) est créé automatiquement, et détruit quand la branche est supprimée.

C'est ici qu'ArgoCD montre sa puissance côté développeur. Lisez attentivement la documentation des ApplicationSet et de leurs *generators* avant de coder — en particulier le pullRequest generator ou le git generator selon ce que vous voulez observer.

### Contraintes.

- utiliser un ApplicationSet (pas une Application répétée 10 fois) ;
- choisir un *generator* explicitement et le justifier : git (sur branches) ou pullRequest (sur PRs ouvertes) ;
- le nom des Application générées suit le pattern <service>-preview-<branche> ;
- le namespace est créé à la volée (option CreateNamespace=true dans la syncOptions) ;
- les ressources doivent porter le label devhub.io/env: preview pour pouvoir être listées en un coup d'œil ;
- **prune obligatoire ici** — sinon les previews ne se nettoient jamais.

### Décomposition suggérée.

1. Lisez les pages *Generators* de la doc ApplicationSet. Faites-vous un avis sur git vs pullRequest.

2. Préparez le Secret qui contient le token GitHub (scope: repo) dans le namespace argocd. **Ne le committez pas.**
3. Écrivez l'ApplicationSet dans platform/apps/preview/<service>.yaml. Inspirez-vous des exemples de la doc, adaptez le template à votre chart.
4. Vérifiez la naming convention : les noms d'Application doivent être des labels K8s valides (pas de slash, pas de majuscules). Utilisez le filter ou une transformation de nom.
5. Poussez une branche feature/demo-prof sur votre repo de service. Attendez 3 minutes (polling). L'Application doit apparaître.

#### **Livrable.**

- un ApplicationSet par service, dans platform/apps/preview/ ;
- une démonstration en direct au formateur : vous créez une branche feature/demo-prof, vous poussez un changement (par exemple replicaCount: 1 ou un message dans /healthz), et vous montrez que dans les 3 minutes, un environnement devhub-preview-feature-demo-prof apparaît dans ArgoCD avec votre changement appliqué ;
- puis vous supprimez la branche : la preview disparaît.

#### **Validation.**

- l'ApplicationSet apparaît dans l'UI, et liste les Application générées en dessous ;
- chaque preview a son propre ingress accessible.

#### **Pièges.**

- oublier que le polling du git generator n'est pas instantané — par défaut 3 minutes. Ne supposez pas que c'est cassé après 30 secondes.
- le pullRequest generator requiert un token GitHub stocké dans un Secret du namespace argocd. Ne le committez pas en clair.
- certains noms de branches ne sont pas des noms K8s valides (slash, majuscules). Le template du nom doit *normaliser* ({{branch\_slug}} ou équivalent).

#### **Défi bonus.**

- utilisez un matrix generator qui croise pullRequest (vos PRs ouvertes) avec une list de 2 régions fictives (fr, eu). Vous obtenez deux previews par PR, une par région. Documentez l'usage métier qu'on pourrait en faire.
- configurez un *webhook* GitHub → ArgoCD pour que les previews apparaissent en quelques secondes au lieu de 3 minutes.

## **PARTIE III — Maîtrise et synthèse**

### **Étape 8 — Drift, rollback, hooks, sync waves**

**Avant (TP 1) / Maintenant (TP 2).** Au TP 1, un `kubectl edit` improvisé en astreinte passait inaperçu — le cluster s'éloignait silencieusement du Git, et personne ne le savait

jusqu'au prochain apply qui écrasait les rustines. Faire un rollback consistait à relancer la CI sur l'ancien commit, en espérant qu'elle marche. Ici, vous allez **provoquer vous-mêmes** un drift et observer comment ArgoCD réagit. Vous allez aussi faire un rollback en *une commande Git* (`git revert`) et chronométrer le temps de convergence. Le contraste est l'objet de cette étape.

**Objectif.** comprendre comment ArgoCD réagit aux écarts, aux échecs, et comment vous contrôlez l'ordre des opérations à la sync.

Réalisez **chaque** des manipulations suivantes et notez ce que vous observez dans `RAPPORT.md`. Pour chacune, prenez une capture de l'UI ArgoCD au moment du diagnostic.

| Manipulation                                                                                                                   | Ce qu'il faut voir                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>kubectl scale deploy annuaire -n devhub-dev --replicas=5</code> sans modifier Git.                                       | ArgoCD passe en <code>OutOfSync</code> . Si <code>selfHeal: true</code> , il redescend à 2.                                                    |
| Pousser un commit qui change <code>image.tag</code> pour un tag qui n'existe pas.                                              | Sync réussit côté Argo, mais le pod reste en <code>ImagePullBackOff</code> . Statut <i>Synced + Degraded</i> .                                 |
| Faire un revert Git du commit précédent.                                                                                       | ArgoCD détecte le nouveau commit, re-sync, le service redevient <i>Healthy</i> . Notez la durée.                                               |
| Ajouter un hook <code>PreSync</code> exécutant une migration de schéma (job qui logge <code>migration ok</code> ).             | À la sync suivante, le job est créé <i>avant</i> le déploiement.                                                                               |
| Ajouter des sync waves : forcer le <code>ConfigMap</code> (wave -1) à être appliqué avant le <code>Deployment</code> (wave 0). | À la sync, l'ordre est respecté. Coupez le <code>ConfigMap</code> (rendez-le invalide) et observez le <code>Deployment</code> ne pas démarrer. |
| Activer <code>prune: true</code> et supprimer un fichier <code>service.yaml</code> du chart.                                   | À la sync, le <code>Service</code> correspondant disparaît du cluster.                                                                         |

### Décomposition suggérée.

1. Faites chaque manipulation dans l'ordre. À chaque fois : capture d'écran, observation, hypothèse, vérification.
2. Pour les hooks et les sync waves, lisez d'abord la page *Resource Hooks* et *Sync Phases and Waves* de la doc ArgoCD. Ce sont deux concepts différents qu'on confond souvent.
3. Mesurez (chrono en main) la durée entre le commit Git et l'application effective dans le cluster. C'est une vraie métrique de plateforme.

**Livrable.** la section *Bestiaire ArgoCD* du rapport, contenant les six scénarios documentés avec captures, observation, hypothèse et conclusion.

**Validation.** vous devez être capable, oralement, de dire : « *si je vois OutOfSync + Degraded, je regarde d'abord X, puis Y, puis Z* ».

**Pièges.**

- oublier qu'un hook PreSync qui échoue **bloque** la sync. C'est voulu. Si vous êtes coincés, regardez les logs du job.
- confondre Sync (ArgoCD) et Rollout (Argo Rollouts, qu'on n'utilise pas ici).

#### Défi bonus.

- écrivez un hook PostSync qui poste un message sur un webhook (un [httpbin.org/post](https://httpbin.org/post) fera l'affaire) avec le nom de l'Application et la version déployée. Documentez comment vous l'industrialiserez pour notifier Slack.
- écrivez une `ignoreDifferences` qui dit à ArgoCD d'ignorer les changements de `replicaCount` faits hors-Git (cas typique : HPA). Démontrez que `kubectl scale` cesse de déclencher un drift.

## Étape 9 — Sécuriser et observer ArgoCD lui-même

**Objectif.** un agent qui a les pleins pouvoirs sur le cluster ne se laisse pas en accès anonyme avec une stack de logs et de métriques laissée par défaut. Cette étape met ArgoCD dans une posture « production-grade côté plateforme » (sans aller jusqu'à la production applicative — c'est l'étape 11).

#### Contraintes.

- RBAC ArgoCD :
  - un rôle `developer` qui voit toutes les Application du AppProject `devhub` mais ne peut sync que celles dont le nom contient son service (regex) ;
  - un rôle `platform-admin` qui a tous les droits.
- Notifications :
  - installer `argocd-notifications` (intégré au chart, à activer) ;
  - sur un échec de sync, envoyer un message sur un webhook (mockez avec <https://webhook.site>) contenant le nom de l'Application, la révision, et l'erreur.
- Observabilité :
  - exposer les métriques Prometheus d'ArgoCD ;
  - dans `RAPPORT.md`, identifier **trois métriques utiles** (avec leur nom exact) et expliquer ce qu'elles vous diraient en cas d'incident.

#### Décomposition suggérée.

1. Lisez la doc RBAC : `argocd-rbac-cm` (ConfigMap) et la syntaxe `p, role:foo, applications, sync, devhub/...`. C'est un mini-DSL.
2. Créez un utilisateur local en plus d'admin (`accounts.<user>: login, apiKey`). Donnez-lui le rôle `developer`. Connectez-vous en tant que lui via la CLI et constatez les restrictions.
3. Pour les notifications, partez de la documentation `argocd-notifications` et des *triggers* prédéfinis (`on-sync-failed`).

4. Pour Prometheus, vérifiez que `argocd-metrics`, `argocd-server-metrics`, `argocd-repo-server` exposent leurs endpoints. Listez les métriques avec `kubectl port-forward + curl`.

#### **Livrable.**

- le fichier de valeurs Helm d'ArgoCD utilisé pour l'install, avec les sections RBAC, notifications et métriques explicitement configurées ;
- une capture du webhook.site recevant une notification d'échec de sync (simulez l'échec en cassant volontairement une Application) ;
- les trois métriques retenues, leur nom, leur unité, et l'interprétation que vous en feriez.

#### **Validation.**

- `argocd app sync <app>` depuis le compte `developer` est *refusé* sur une Application qui ne lui appartient pas, *accepté* sur la sienne ;
- une sync qui échoue produit une notification visible dans `webhook.site`.

#### **Pièges.**

- la regex RBAC d'ArgoCD utilise `glob` par défaut, pas regex POSIX. Lisez la doc avant de vous arracher les cheveux.
- le ConfigMap `argocd-rbac-cm` peut nécessiter un redémarrage du `argocd-server` pour être rechargé (selon version).

**Défi bonus.** connectez l'authentification d'ArgoCD à un **OIDC** local. Solution la plus simple : lancez un `dex` en mode statique avec deux utilisateurs (un dev, un admin), et configurez ArgoCD pour s'y adosser. Documentez les *claims* qui mappent vers vos rôles RBAC.

## **Étape 10 — Comparer les outils GitOps (*bonus, au-delà des 7h*)**

**Objectif.** prendre du recul sur ArgoCD en le comparant explicitement à son principal concurrent (Flux) et à un mode 100 % CI (Helm + GitHub Actions sans GitOps).

**Livrable.** dans `RAPPORT.md`, une matrice à trois colonnes, avec **votre note argumentée** (0 à 5) pour chaque critère :

| Critère                                              | ArgoCD | Flux | Helm + Actions (sans GitOps) |
|------------------------------------------------------|--------|------|------------------------------|
| Courbe d'apprentissage                               | ...    | ...  | ...                          |
| UI prête à l'emploi pour devs                        | ...    | ...  | ...                          |
| Adapté à un mono-repo                                | ...    | ...  | ...                          |
| Adapté à 50 repos                                    | ...    | ...  | ...                          |
| Coût opérationnel (CPU/RAM dans le cluster)          | ...    | ...  | ...                          |
| Maturité du multi-cluster                            | ...    | ...  | ...                          |
| Disponibilité d'extensions (Rollouts, Notifications) | ...    | ...  | ...                          |
| Risque opérationnel si l'agent tombe                 | ...    | ...  | ...                          |

**Validation.** restituez la matrice à voix haute, sans la relire. Le formateur vous chronomètre — 5 minutes max. C'est un exercice de **prise de parole technique**, pas de copie.

## Étape 11 — Synthèse obligatoire : « *ArgoCD, et la prod alors ?* »

**Objectif.** cette étape est la plus importante du TP. C'est elle qui distingue « *j'ai cliqué sur Sync* » de « *je comprends GitOps* ». Elle ne produit pas de code — elle produit du raisonnement.

**Livrable 1 — Rétrospective TP 1 → TP 2.** reprenez le tableau du chapitre « *Le même geste, deux paradigmes* » et **commentez chaque ligne** dans `RAPPORT.md` :

- en une phrase, qu'avez-vous *vraiment ressenti* en réalisant l'opération avec ArgoCD (plus rapide ? plus contraint ? plus rassurant ?) ;
- identifiez **au moins deux opérations** où la version *ArgoCD* est **plus contraignante** que la version TP 1, et expliquez pourquoi cette contrainte est néanmoins justifiée (ou pas — votre prise de position est libre, mais elle doit être argumentée) ;
- identifiez **l'opération** qui, à elle seule, justifierait à vos yeux d'adopter ArgoCD si vous ne deviez en garder qu'une.

**Livrable 2 — Ce qu'ArgoCD ne sait pas faire.** un chapitre de `RAPPORT.md` structuré ainsi :

Pour chacun des sept thèmes ci-dessous, expliquez en 5 à 10 lignes :

1. **Le risque concret** que vous prenez si vous déployez DevHub Campus en l'état chez un vrai client.
2. **L'outil complémentaire** que vous installeriez en plus d'ArgoCD pour traiter le risque.
3. **Une référence** (doc officielle, article, vidéo) que vous garderiez sous la main.

Thèmes à traiter :

| Thème                                       | Mot-clé à creuser                                                  |
|---------------------------------------------|--------------------------------------------------------------------|
| Déploiement progressif (canary, blue/green) | Argo Rollouts, Flagger                                             |
| Validation des manifests avant sync         | Kyverno, OPA Gatekeeper, Conftest                                  |
| Gestion des secrets dans Git                | Sealed Secrets, External Secrets Operator, SOPS                    |
| Signature et provenance des images          | cosign, Sigstore, admission policies                               |
| RBAC multi-équipe sur ArgoCD                | AppProject, SSO/OIDC, RBAC CSV                                     |
| Disaster recovery applicatif                | Velero, snapshots de PVC, dump SGBD                                |
| Multi-cluster                               | Cluster Inventory, ApplicationSet cluster generator, hub-and-spoke |

**Validation.** restituez à voix haute, devant le formateur, votre prise de position en deux minutes : « *si demain je deviens responsable de la plateforme DevHub Campus, voici les trois briques que j'ajoute en priorité après ArgoCD, et pourquoi.* »

## **Boss final (bonus, pour les binômes qui terminent largement en avance)**

**Objectif.** vous êtes en astreinte. Le formateur va casser **une seule chose** dans votre plateforme. Charge à vous de la diagnostiquer et de la réparer **sans tout détruire**.

### **Règles du jeu.**

1. Vous avancez de l'étape 1 à 9. À ce stade, votre plateforme tourne *Healthy*.
2. Vous prévenez le formateur. Il intervient (5 min) pour casser **un seul élément** : un commit Git, un `kubectl edit`, une suppression de ressource, un changement de RBAC, un secret modifié, etc.
3. Le formateur ne vous dit pas ce qu'il a fait.
4. Vous avez **20 minutes** pour : (a) caractériser le symptôme, (b) localiser la cause, (c) la corriger **uniquement via Git** (pas de `kubectl edit` pour réparer).
5. À la fin, vous expliquez au formateur : *quelle hypothèse vous avez écartée, et pourquoi celle que vous avez retenue collait.*

**Critère de réussite.** au moins l'une de ces trois conditions :

- vous trouvez et réparez en moins de 20 min ;
- vous ne trouvez pas en 20 min **mais** vous avez bien posé le diagnostic (cause identifiée, plan d'action écrit) ;
- vous ne trouvez pas mais vous avez expliqué proprement comment vous *préviendriez* ce type d'incident à l'avenir.

**Note.** cet exercice n'est pas évalué — il est là pour vous tester. Ce que vous y vivez vaut tout le reste du TP.

## Annexe A — Pour aller plus loin

Les sujets ci-dessous **ne sont pas évalués** mais ouvrent la suite logique du TP. Si vous finissez avant la fin, choisissez-en un et tentez-le.

| Extension                                               | Ce que vous y gagnez                                            |
|---------------------------------------------------------|-----------------------------------------------------------------|
| Brancher Argo Rollouts sur l'un des services            | Voir un vrai déploiement canary avec analyse de métriques.      |
| Mettre en place Sealed Secrets                          | Pousser un Secret chiffré dans Git sans frémir.                 |
| Activer l'authentification GitHub OAuth sur ArgoCD      | UI personnelle, RBAC par équipe.                                |
| Adopter Kustomize au lieu de Helm sur un service        | Sentir la différence template / overlay.                        |
| Brancher un webhook Git → ArgoCD                        | Tomber sous les 5 secondes de réactivité.                       |
| Intégrer Kyverno + une policy ArgoCD-aware              | Refuser une sync qui violerait une règle (ex : pas de :latest). |
| Passer en multi-cluster (cluster dev + cluster staging) | Comprendre ApplicationSet cluster generator.                    |

## Annexe B — Pièges fréquents

| Symptôme                                             | Cause la plus probable                                                                                                                                            |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L'Application reste OutOfSync après sync             | Un champ est <i>défini par défaut</i> par K8s mais absent du chart (ex. imagePullPolicy). Ajoutez-le explicitement, ou whitelisez le champ via ignoreDifferences. |
| helm template fonctionne en local mais ArgoCD échoue | La version de Helm utilisée par argocd-repo-server diffère de la vôtre. Forcer la version dans la source ou aligner.                                              |
| Le namespace n'est pas créé                          | syncOptions: CreateNamespace=true manquant.                                                                                                                       |
| Un secret rempli côté UI disparaît après sync        | Vous l'avez créé hors-Git ; ArgoCD prune. C'est la promesse GitOps : <i>si ce n'est pas dans Git, ça n'existe pas</i> .                                           |
| L'ApplicationSet ne génère rien                      | Token de l'API Git absent, ou regex de branches qui ne matche aucune branche. Vérifier la section Status de l'ApplicationSet.                                     |
| Les previews ne se suppriment pas                    | prune: true manquant sur l'ApplicationSet.                                                                                                                        |
| L'UI ArgoCD est lente sur Mac/Apple Silicon          | Le argocd-repo-server n'a pas d'images ARM dans certaines versions ; vérifier la nodeSelector ou bumper la version.                                               |
| Un commit Git ne déclenche pas de sync               | Polling de 3 min ; activez un webhook ou diminuez timeout.reconciliation.                                                                                         |



## Annexe C — Critères d'évaluation

| Critère                                                                                            | Poids |
|----------------------------------------------------------------------------------------------------|-------|
| Qualité de l'image Docker (taille, non-root, healthcheck, tag)                                     | 8 %   |
| Qualité du chart Helm (paramétrage, helpers, multi-env)                                            | 12 %  |
| Installation propre d'ArgoCD (chart, ingress, rotation mdp)                                        | 8 %   |
| Pattern App of Apps et AppProject sécurisé                                                         | 14 %  |
| ApplicationSet et previews fonctionnels (démon en direct)                                          | 18 %  |
| Bestiaire ArgoCD (étape 8, drift/rollback/hooks/waves documentés)                                  | 12 %  |
| Sécurité et observabilité d'ArgoCD (étape 9 : RBAC, notif, métriques)                              | 12 %  |
| Chapitre de synthèse « <i>Ce qu'ArgoCD ne sait pas faire</i> » (étape 11)                          | 12 %  |
| Lisibilité du RAPPORT.md (structure, schémas, captures)                                            | 4 %   |
| <b>Mention possible si</b> : étape 10 traitée, ou Boss final tenté, ou $\geq 4$ défis bonus rendus | —     |